```
            context.log.info(
              s"I'm overloaded. Counting '$current' while
              ➥max is '$max'. Stopping")
            Behaviors.stopped                                    ❸
          }
            case Decrease(amount) =>                             ❹
              ...

      }
    }
}
```

❶ Input, total, and max representing the state

❷ Output as a behavior, not as a call to a method

❸ Stops the actor. Any message left in the mailbox will not reach the actor.

❹ The same as Increase, but with the obvious differences

Remember, when the wallet increases and the current value is less than the maximum value, the wallet is further increased by setting `WalletState.apply(current, max)` as the next behavior. From the first to the second message, the behaviors are first `.apply(0,2)` and then `.apply(0 + 1, 2)`.

Conversely, `Decrease` stops when the wallet value becomes negative. The code is practically the same, so we won't repeat it here. If you are curious, you can look at the code in the repo.

Note how the protocol is written within the actor. The messages of a protocol are always specific to an actor, and in this sense, they are not meant to be reused by other actors. They are the API of the actor and belong to it.

In this section, you have learned how to manage state as a value with an actor. You need to add the state in its constructor and pass it to the next behavior. But there is also another way to store state in an actor: with behaviors.